# COMP 3613 Software Engineering II

## Assignment 2 - Student Awards Application Test Report

**Student name:** Edan Ali **Student ID:** 816039649 **Project:** Student Awards **Repository:** al5428/comp3613a1 - assignment2 branch **Report date:** 7 November 2026

## Contents

1. A. Project overview and workflows
2. B. Design artifacts
3. C. Test plans
4. D. Test implementation
5. E. Test reports and results
6. F. Conclusion and limitations
7. Appendix A. Test artifact locations

# A. Project overview and workflows

## A.1 Project description

The Student Awards application is a web-based volunteer recognition system. It has two primary roles:

- **Students** submit volunteer activities with the activity name, hours worked, activity date, description, and supporting evidence. A new submission is pending review. Students can view their approved-hour total, submission history, milestone progress, and award status.
- **Administrators** review submitted activity details and evidence, then approve or reject volunteer-hour submissions. They also review award redemption requests.

Only approved volunteer hours contribute to a student's progress. The application tracks three thresholds: **Plaque (25 hours)**, **Medal (50 hours)**, and **Trophy (100 hours)**. When eligible, a student can request the corresponding award. Redemption requests are reviewed by an administrator; the application prevents a student from requesting the same award more than once.

## A.2 Main workflows

The project's three main workflows are:

1. Verify Volunteer Hours
2. Track Progress and Unlock Milestones
3. Redeem an Award

# B. Design artifacts

## B.1 Use Case Diagram

The supplied diagram shows the Student and Administrator actors, the three main workflows, and the supporting evidence and milestone-detail use cases.

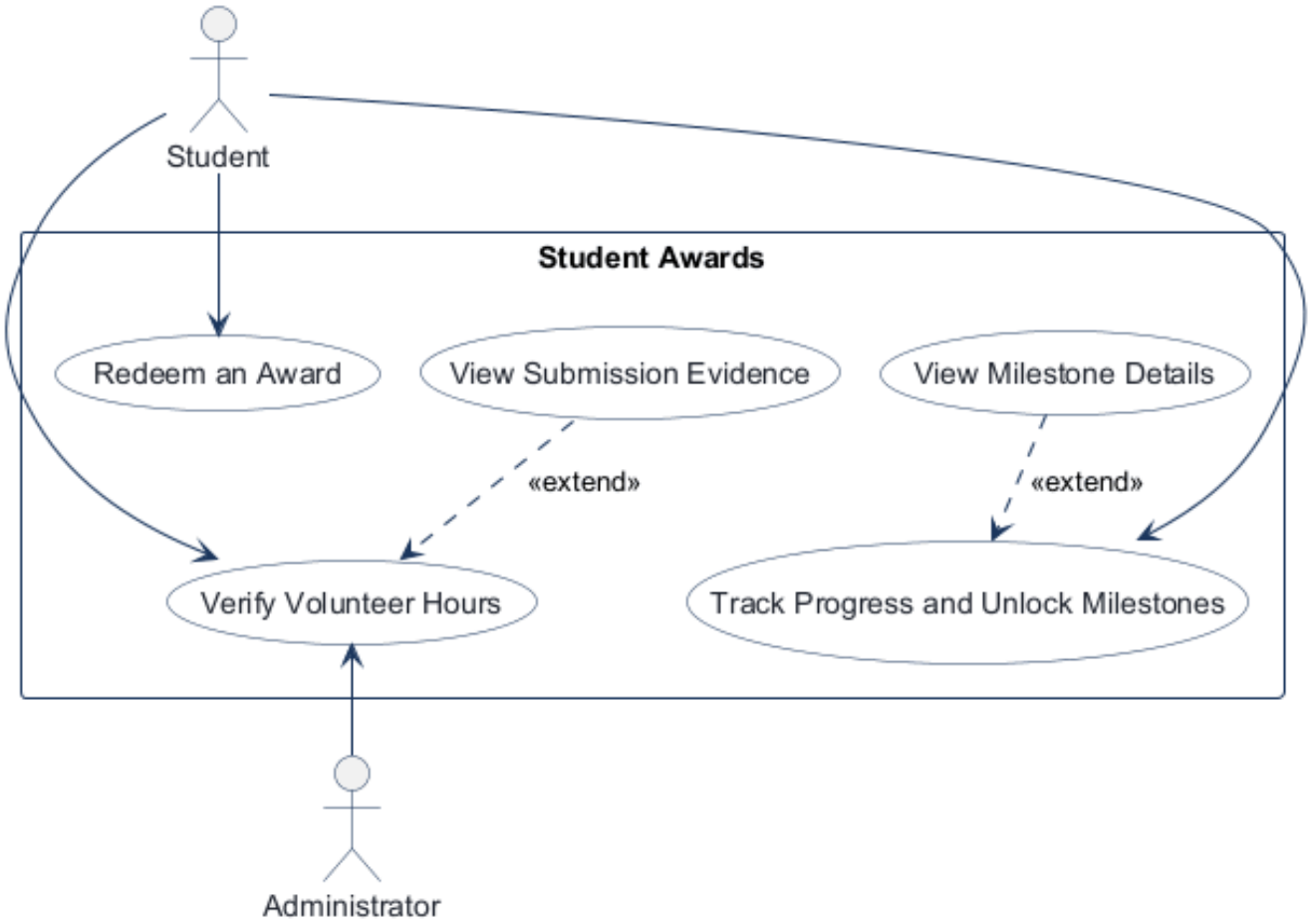



**Figure 1.** Student Awards use case diagram supplied for Assignment 2.

## B.2 Entity Relationship Diagram (ERD)

The supplied ERD shows Student, Administrator, Volunteer Submission, Milestone, Student Milestone, Award, and Redeemed Award entities and their relationships.

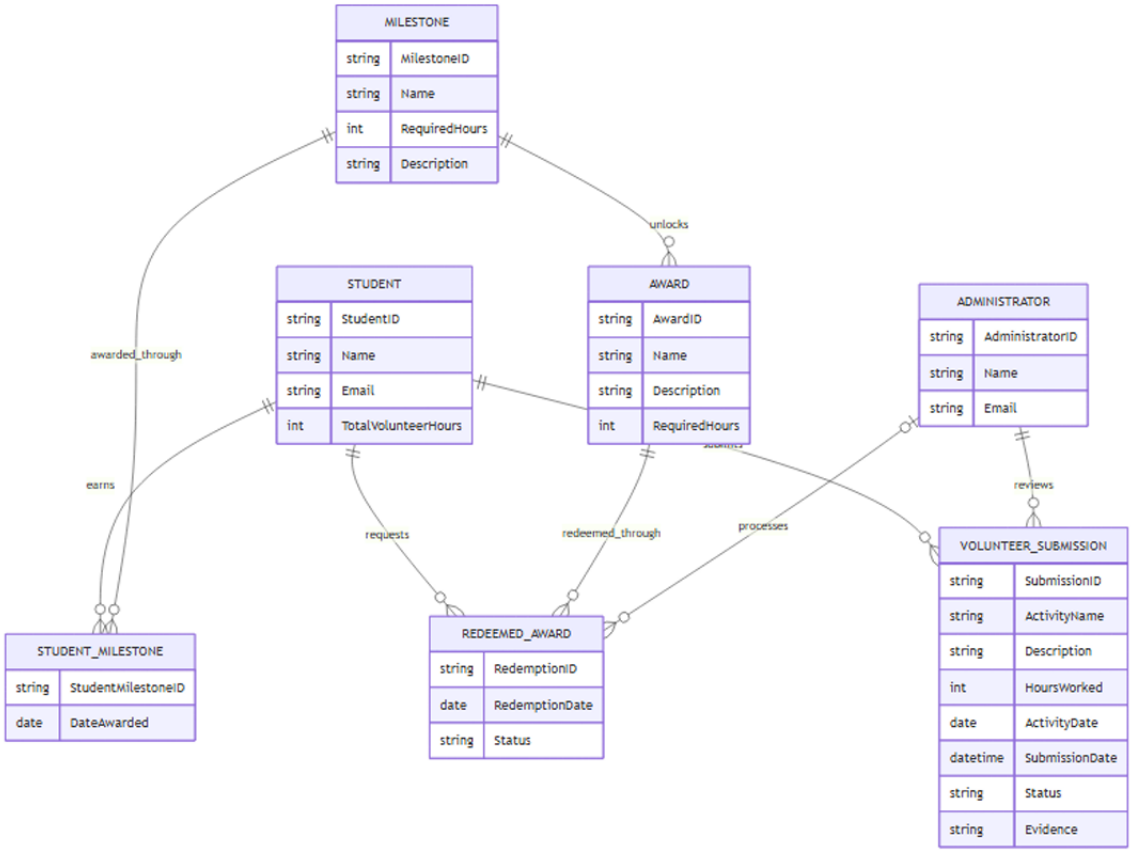



**Figure 2.** Student Awards ERD supplied for Assignment 2.

## B.3 Application Wireframes

The wireframe image covers student volunteer-hour submission, administrator review, progress and milestone details, and award redemption.




**Figure 3.** Student Awards application wireframes supplied for Assignment 2.

# C. Test plans

## C.1 Unit Test Plan

### 1. Objective

Verify that individual methods in VolunteerService handle validation, approved-hour calculations, and award redemption rules correctly without using a real database.

### 2. Testing Approach

Each test targets one method. Repository and database dependencies will be replaced with mocks so tests are independent and repeatable.

### 3. Unit Test Cases

The table below documents the eight existing tests in tests/test_volunteer_service_unit.py, including their test inputs and asserted behavior.

| Unit Test | Input | Expected Output |
|---|---|---|
| test_empty_activity_name | Call create_submission(student_id=1, activity_name="  ", hours_worked=1, evidence=""). | Raises ValueError with message "Activity name is required."; repo.create is not called. |
| test_zero_volunteer_hours | Call create_submission(student_id=1, activity_name="Campus garden", hours_worked=0, evidence=""). | Raises ValueError with message "Hours worked must be greater than zero."; repo.create is not called. |
| test_approved_hours | For student 1, mock two approved submissions with hours_worked=5 and hours_worked=10; call get_total_approved_hours(student_id=1). | Returns 15; repo.list_approved_for_student is called once with 1. |
| test_invalid_review_decision | Call review_submission(submission_id=1, decision="pending"). | Raises ValueError with message "Decision must be approved or rejected."; repo.update_status and administrator_repo.get_or_create_for_user are not called. |
| test_ineligible_redemption | For student 1, mock synchronized hours as 20 and award 2 as requiring 25 hours; call redeem_award(student_id=1, award_id=2). | Raises ValueError with message "Student does not yet meet the hour requirement for this award."; the redemption lookup and redemption_repo.create are not called. |
| test_duplicate_redemption | For student 1, mock synchronized hours as 25, award 2 as requiring 25 hours, and an existing redemption; call redeem_award(student_id=1, award_id=2). | Raises ValueError with message "This award has already been requested and cannot be redeemed again."; get_by_student_and_award is called once with (1, 2) and redemption_repo.create is not called. |
| test_award_eligibility | For student 1, mock 25 approved hours and no existing redemption. Return award 2 named "Medal", description "A 50-hour award", requiring 50 hours and linked to milestone 3, named "Community Builder". | Returns [{"id": 2, "name": "Medal", "description": "A 50-hour award", "required_hours": 50, "milestone_name": "Community Builder", "eligible": false, "redeemed": false, "status": "not_requested", "hours_remaining": 25}]; hour synchronization and approved-hour lookup are each called once with 1, and the redemption lookup is called once with (1, 2). |
| test_invalid_redemption_review | Call review_redemption(redemption_id=1, decision="cancelled"). | Raises ValueError with message "Decision must be approved or rejected."; redemption_repo.update_status and administrator_repo.get_or_create_for_user are not called. |

### 4. Entry Criteria

- The Student Awards application source code is available.
- pytest and project dependencies are installed.
- Mocks can be configured for service dependencies.

## C.2 Integration Test Plan

### 1. Objective

Verify that the Student Awards application's services, repositories, models, and database work together to support its main workflows.

### 2. Testing Approach

The existing tests create an isolated in-memory SQLite database. They exercise real application components without changing the deployed PostgreSQL database.

### 3. Integration Test Cases

The tests call service and CLI helper functions directly; they do not exercise HTTP routes.

| Test | Dependencies | Description |
| --- | --- | --- |
| test_submit_and_review_flow | In-memory SQLite create_engine, SQLModel.metadata.create_all, Session; add_reviewer; VolunteerService.create_submission() and review_submission(); VolunteerSubmissionRepository.create() and update_status(); AdministratorRepository.get_or_create_for_user(); Student Awards and User models. | A submission for 4 volunteer hours is created with pending status and a non-null activity date. Approving it with the reviewer records approved status and that reviewer's administrator ID. |
| test_student_service_resolves_user_to_different_student_id | In-memory SQLite create_engine, SQLModel.metadata.create_all, Session; VolunteerService.get_or_create_student_for_user(), create_submission_for_user(), and get_student_summary(); StudentRepository.get_or_create_for_user() and get_by_user_id(); VolunteerSubmissionRepository.create(), list_for_student(), and list_approved_for_student(); User, Student, and VolunteerSubmission models. | A user with ID 13 is associated with a pre-existing Student record with ID 11. The service resolves that student, saves the new submission with student_id=11, and reports a submission count of 1 for the student. |
| test_milestone_unlock_and_award_redemption | In-memory SQLite create_engine, SQLModel.metadata.create_all, Session; add_reviewer; VolunteerService.create_submission(), review_submission(), get_or_create_student(), sync_student_hours(), list_milestones_for_student(), redeem_award(), review_redemption(), and list_awards_for_student(); VolunteerSubmissionRepository, AdministratorRepository, StudentRepository, MilestoneRepository, AwardRepository, StudentMilestoneRepository, and RedeemedAwardRepository; Student Awards and User models. | Approving a 10-hour submission unlocks the 10-hour milestone and associates the campus-merch award with it. The eligible student can request that award; its redemption starts pending and can be marked rejected by the reviewer. The award is then shown as redeemed with rejected status, and a second request raises ValueError. |

| Test | Dependencies | Description |
|---|---|---|
| test_demo_seed_data_is_consistent_and_rerunnable | In-memory SQLite create_engine, SQLModel.metadata.create_all, Session, session.exec(select(...)); app.cli._seed_demo_data(); UserRepository.get_by_username() and create(); StudentRepository.get_or_create_for_user(); AdministratorRepository.get_or_create_for_user(); VolunteerService.get_student_summary(), list_for_review(), list_submissions_for_admin(), get_submission_detail_for_admin(), and list_pending_redemptions(); verify_password(); Student Awards and User models. | The first seed call reports 13 created and 0 skipped. The seeded users, students, administrators, submissions, milestones, awards, student milestones, and redemptions meet the asserted counts and consistency rules. After usernames and passwords are changed to legacy values and seeding runs again, there are still 30 submissions and 6 redemptions, and the student credentials and usernames are restored as asserted. |
| test_legacy_bob_account_is_migrated_without_losing_student_history | In-memory SQLite create_engine, SQLModel.metadata.create_all, Session, session.exec(select(...)); app.cli._seed_demo_data(); UserRepository.get_by_username(); StudentRepository.get_or_create_for_user(); encrypt_password() and verify_password(); User and Student models. | Seeding a legacy bob user with a Bruce Wayne student record preserves the original user and student IDs while renaming that account to Bruce Wayne. A separate bob user and Bob student are created, with the expected student names and passwords for both accounts. |
| test_seed_adds_description_column_to_legacy_submission_table | In-memory SQLite create_engine; engine.begin(); SQLAlchemy text() and inspect(); app.cli._ensure_submission_description_column(). | Given a legacy volunteersubmission table containing only id, the helper adds a column named description. |
| test_seed_adds_and_backfills_activity_date_column | In-memory SQLite create_engine; engine.begin() and engine.connect(); SQLAlchemy text() and inspect(); app.cli._ensure_submission_activity_date_column(). | Given a legacy submission with submission_date='2026-09-15 12:30:00' and no activity_date column, the helper adds the column and backfills the row's activity date to 2026-09-15. |

### 4. Entry Criteria

- Application models, services, and repositories are available.
- pytest and database dependencies are installed.
- The test database can be initialized.

## C.3 User Acceptance Test Plan

### 1. Objective

Confirm that students and administrators can complete the main tasks shown in the Student Awards use cases and wireframes.

### 2. Test Environment

| Tester Information | Details |
|---|---|
| Tester Name | Edan Ali |
| Tester Position/Role | Student Developer |
| Date of Test (DD/MM/YYYY) | 07/11/2026 |
| **Item** | **Details** |
| Application | Student Awards web application |
| Platform | Web browser |
| Environment | Deployed application |
| Test data | Demo accounts, volunteer submissions, milestones, and awards; individual account credentials are not recorded here. |

### 3. Acceptance Test Cases

The table records the eight existing Student Awards UAT scenarios in execution order. Each case links to its corresponding screenshot in docs/assignment2/screenshots/.

| Test Case | Pre-conditions | Test Steps | Test Criteria | Success (Pass/Fail) |
|---|---|---|---|---|
| UAT01 - Submit volunteer hours | Student is signed in to the deployed Student Awards application and can access Volunteer Hours. | 1. Open Volunteer Hours.<br>2. Enter the recorded activity Community Cleanup and 6 hours, with the activity details required by the form.<br>3. Submit the activity for review. | The submission history shows Community Cleanup, 6h, and Pending.<br>Evidence: UAT01 screenshot | Pass - recorded |
| UAT02 - Admin approves volunteer hours | Administrator is signed in, and a pending Community Cleanup submission for 6 hours is available for review. | 1. Open the administrator's Volunteer Hours list.<br>2. Locate the Community Cleanup entry showing 6 hours.<br>3. Approve the submission. | The page confirms the submission was marked approved, and the administrator's list shows Community Cleanup, 6 hours, and Approved.<br>Evidence: UAT02 screenshot. The screenshot matches UAT01 by activity name and hours but does not show a submission ID to confirm it is the same record. | Pass - recorded |
| UAT03 - Admin rejects volunteer hours | Administrator is signed in, and the Beach Cleanup submission for 3 hours is pending review. | 1. Open the administrator's Volunteer Hours list.<br>2. Locate Beach Cleanup, showing 3 hours.<br>3. Reject the submission. | The page confirms the submission was marked rejected, and the list shows Beach Cleanup, 3 hours, and Rejected.<br>Evidence: UAT03 screenshot | Pass - recorded |
| UAT04 - View approved volunteer hours total | Student Bob is signed in with the recorded progress snapshot: 3 submissions, including approved hours and a rejected 3-hour Beach Cleanup submission. | 1. Open the student Dashboard.<br>2. Read the Approved, Pending, and Submissions summary values.<br>3. Compare the approved total with the rejected Beach Cleanup entry. | The dashboard shows 11h Approved, 0 Pending, and 3 Submissions; the rejected 3 hours are not included in the approved total. This is the earlier progress snapshot, before the additional approved hours shown in UAT05.<br>Evidence: UAT04 screenshot | Pass - recorded |
| UAT05 - Unlock milestone plaque at 25 approved hours | Student Bob is signed in after additional approved hours have brought the recorded total to 25. | 1. Open the student Dashboard.<br>2. Read the approved-hour total and the Plaque Milestone card. | The dashboard shows 25h Approved; the Plaque Milestone shows Unlocked, 25 / 25 approved hours, and 100% progress.<br>Evidence: UAT05 screenshot | Pass - recorded |
| UAT06 - Submit milestone redemption request | Student is signed in, has at least 25 approved hours, and has not already requested the Plaque award. | 1. Open Awards.<br>2. Locate Plaque, which requires 25 approved hours.<br>3. Request the Plaque award redemption. | The Awards page confirms the request and displays the Plaque award with pending status.<br>Evidence: UAT06 screenshot | Pass - recorded |
| UAT07 - Prevent duplicate redemption | Student is signed in and already has a pending redemption request for the Plaque award. | 1. Open Awards after submitting the existing request.<br>2. Inspect the Plaque award and available actions. | The Plaque award remains pending, and no second redemption action is available through the student interface.<br>Evidence: UAT07 screenshot | Pass - recorded |

| Test Case | Pre-conditions | Test Steps | Test Criteria | Success (Pass/Fail) |
|---|---|---|---|---|
| UAT08 - Admin approves redemption request | Administrator is signed in, and Bob's Plaque award redemption request is available for review. | 1. Open the administrator's award redemption review page.<br>2. Locate Bob's Plaque request.<br>3. Approve the redemption. | The Awards page displays the Plaque award with approved status after review.<br>Evidence: UAT08 screenshot | Pass - recorded |

## 4. Acceptance Criteria

A test passes when the tester can perform the stated steps and observe the expected result in the application interface without an unexpected error.

## 5. UAT Results Record

The following results were recorded after executing all eight User Acceptance Tests on the deployed Student Awards application.

| Test ID | Actual Result | Pass / Fail | Comments |
|---|---|---|---|
| UAT01 | 6-hour Community Cleanup submission displayed with Pending status. | ☑ Pass | The updated UAT01 screenshot shows 6 hours and Pending status. |
| UAT02 | Administrator approved the 6-hour Community Cleanup submission, and its status changed to Approved. | ☑ Pass | The screenshot matches UAT01's activity name and 6-hour value; no submission ID is visible to confirm it is the same record. |
| UAT03 | Administrator rejected Beach Cleanup, and its status changed to Rejected. | ☑ Pass | Successfully rejected the 3-hour submission. |
| UAT04 | Dashboard displayed 11 approved hours, excluding the rejected 3-hour submission. | ☑ Pass | This is the progress snapshot before the additional approved hours recorded by UAT05. |
| UAT05 | After additional approved hours, the student reached 25 approved hours and the Plaque Milestone displayed Unlocked. | ☑ Pass | The later dashboard shows 25 approved hours and the Plaque Milestone unlocked. |
| UAT06 | Plaque Award redemption request displayed Pending status. | ☑ Pass | Student successfully requested the Plaque Award after reaching 25 approved hours. |
| UAT07 | Plaque Award displayed Pending status with no option to request redemption again. | ☑ Pass | Duplicate redemption prevented through the student interface. |
| UAT08 | Administrator approved Bob's Plaque Award redemption, and its status changed to Approved. | ☑ Pass | Award redemption successfully reviewed and approved. |

## 6. UAT Summary

All 8 User Acceptance Tests passed successfully. The Student Awards application supported volunteer-hour submission, administrator approval and rejection, student progress tracking, milestone unlocking, award redemption, and prevention of duplicate redemption through the student interface.

**Overall UAT Result: 8/8 Passed (100%)**

# D. Test implementation

## D.1 Unit test implementation

The isolated unit tests are in tests/test_volunteer_service_unit.py. They test existing VolunteerService methods using pytest and unittest.mock.Mock; the fixture supplies a mock database object and replaces repository dependencies with mocks. The tests do not create SQLModel sessions or connect to a database.

The eight tests cover:

1. test_empty_activity_name - rejects a blank activity name and verifies the submission repository is not called.
2. test_zero_volunteer_hours - rejects zero hours and verifies the submission repository is not called.
3. test_approved_hours - totals mocked approved submissions of 5 and 10 hours as 15.
4. test_invalid_review_decision - rejects pending as a review decision and verifies no update call is made.
5. test_ineligible_redemption - rejects redemption at 20 hours for a 25-hour award and verifies no redemption lookup or creation occurs.
6. test_duplicate_redemption - rejects a second request for an already-requested award and verifies no new redemption is created.
7. test_award_eligibility - checks that 25 hours is insufficient for a 50-hour award and that 25 hours remain.
8. test_invalid_redemption_review - rejects cancelled as a decision and verifies no redemption update is made.

## D.2 Integration test implementation

The integration tests are in tests/test_student_awards_flow.py. They use real application models, services, repositories, and helper functions with an in-memory SQLite engine. This covers database-backed interactions while avoiding the deployed PostgreSQL database.

The seven integration tests cover submission and review, user-to-student ID resolution, milestone unlock and award redemption, repeatable demo seeding, legacy account migration, adding the legacy description column, and adding/backfilling the activity_date column. The seed test also checks record counts and consistency across students, submissions, milestones, awards, and redemptions.

## D.3 Postman API test implementation

The exported Postman collection is COMP3613 Assignment 2 - Student Awards API Tests.postman_collection.json. It contains ten **GET-only** requests. Its assertions cover:

- Health-check status and JSON health value.
- Login, home, and registration page status/content type.
- Denial of unauthenticated access to student and administrator pages.
- Rejection of a milestone path parameter that is not an integer.

The collection does not contain POST, PUT, PATCH, or DELETE requests; its run therefore does not verify state-changing API operations. The supplied Postman screenshots show a single iteration with **10 requests, 21 assertions passed, 0 failed, and 0 errors** against the deployed service.

## D.4 Pytest CLI commands

Run these commands from the repository root in Windows PowerShell. They target the two test files represented in the supplied pytest results:

```
.\.venv\Scripts\python.exe -m pytest tests\test_volunteer_service_unit.py -q
.\.venv\Scripts\python.exe -m pytest tests\test_student_awards_flow.py -q
```

To run both suites together:

```
.\.venv\Scripts\python.exe -m pytest tests\test_volunteer_service_unit.py tests\test_student_awards_flow.py -q
```

## E. Test reports and results

## E.1 Unit test result

The supplied pytest result screenshot shows **8 tests passed** for tests/test_volunteer_service_unit.py.

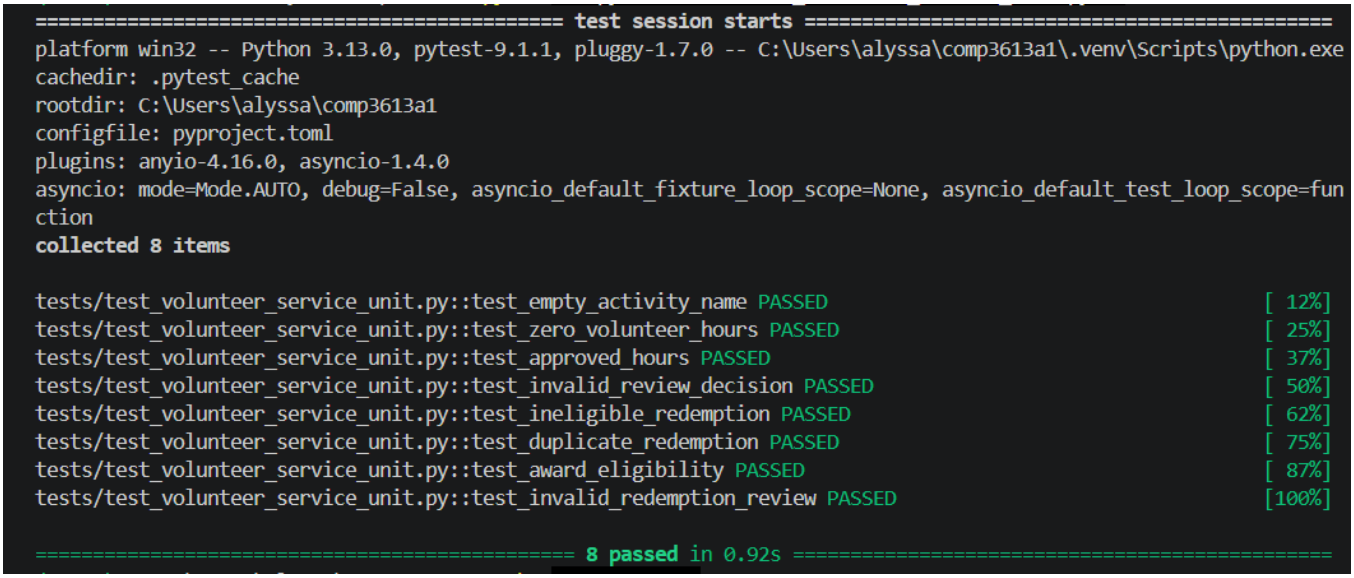



**Figure 4.** Unit test run showing all 8 planned VolunteerService unit tests passed.

## E.2 Integration test result

The supplied pytest result screenshot shows **7 tests passed** for tests/test_student_awards_flow.py. It also shows one SQLAlchemy/SQLite deprecation warning; the warning did not cause a test failure.

```
============================================= test session starts =============================================
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\alyssa\comp3613a1\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\alyssa\comp3613a1
configfile: pyproject.toml
plugins: anyio-4.16.0, asyncio-1.4.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 7 items

tests/test_student_awards_flow.py::test_submit_and_review_flow PASSED                                [ 14%]
tests/test_student_awards_flow.py::test_student_service_resolves_user_to_different_student_id PASSED [ 28%]
tests/test_student_awards_flow.py::test_milestone_unlock_and_award_redemption PASSED                 [ 42%]
tests/test_student_awards_flow.py::test_demo_seed_data_is_consistent_and_rerunnable PASSED           [ 57%]
tests/test_student_awards_flow.py::test_legacy_bob_account_is_migrated_without_losing_student_history PASSED [ 71%]
tests/test_student_awards_flow.py::test_seed_adds_description_column_to_legacy_submission_table PASSED [ 85%]
tests/test_student_awards_flow.py::test_seed_adds_and_backfills_activity_date_column PASSED          [100%]

============================================== warnings summary ===============================================
tests/test_student_awards_flow.py::test_seed_adds_and_backfills_activity_date_column
  C:\Users\alyssa\comp3613a1\.venv\Lib\site-packages\sqlalchemy\engine\default.py:964: DeprecationWarning: The default dat
e adapter is deprecated as of Python 3.12; see the sqlite3 documentation for suggested replacement recipes
    cursor.execute(statement, parameters)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========================================== 7 passed, 1 warning in 6.35s ===========================================
```

**Figure 5.** Integration test run showing 7 tests passed and one deprecation warning.

## E.3 Postman collection run

The first screenshot shows the Postman run summary and the first requests; the second continues the request list. Together they show the completed ten-request collection and the **21/21 passing assertions**.

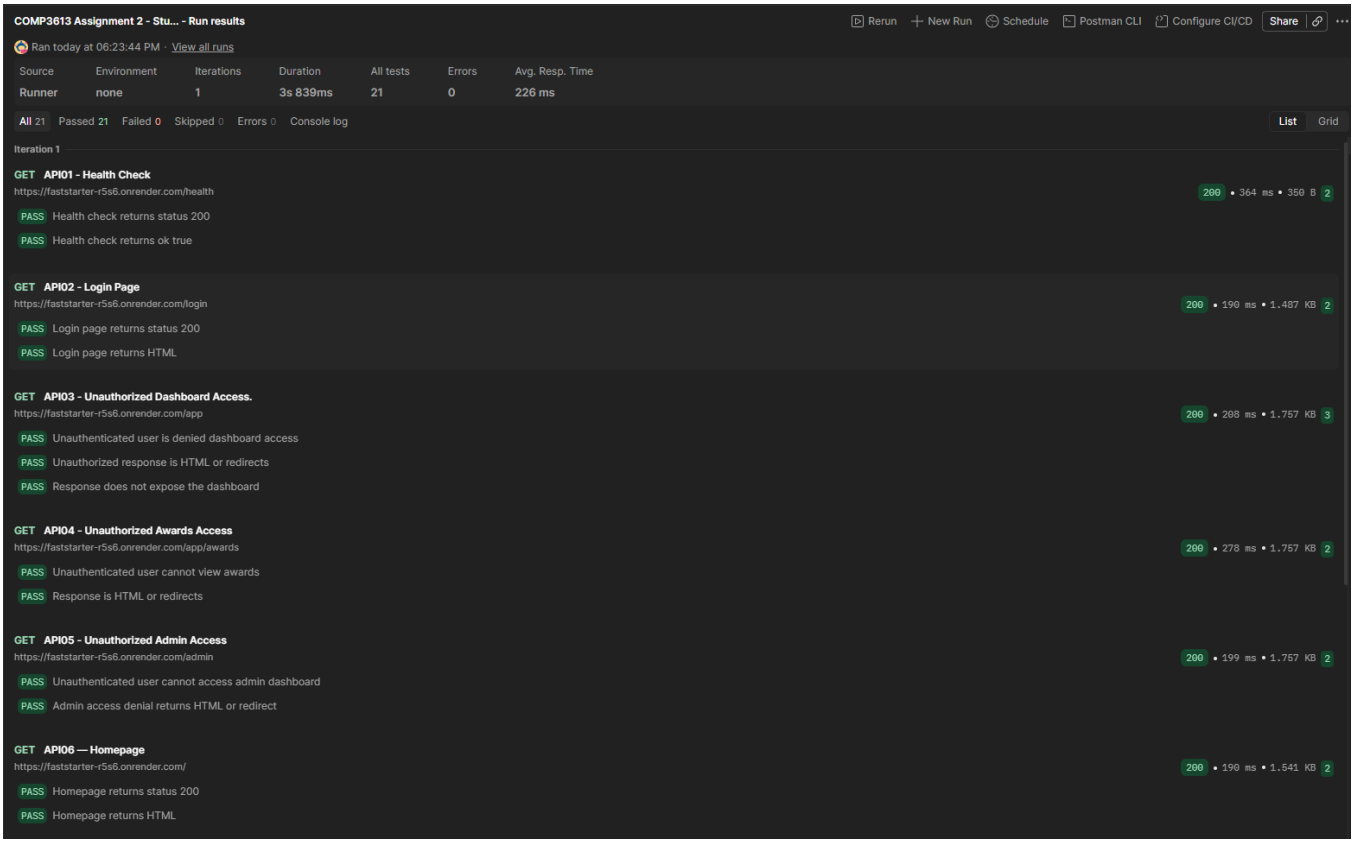



**Figure 6.** Postman run summary and API01-API06 results.




**Figure 7.** Postman API07-API10 results, including registration, protected-route, and invalid milestone checks.

## E.4 User Acceptance Test evidence

The UAT plan records all eight acceptance tests as passed. Each supplied screenshot below is included as evidence for its corresponding test case.

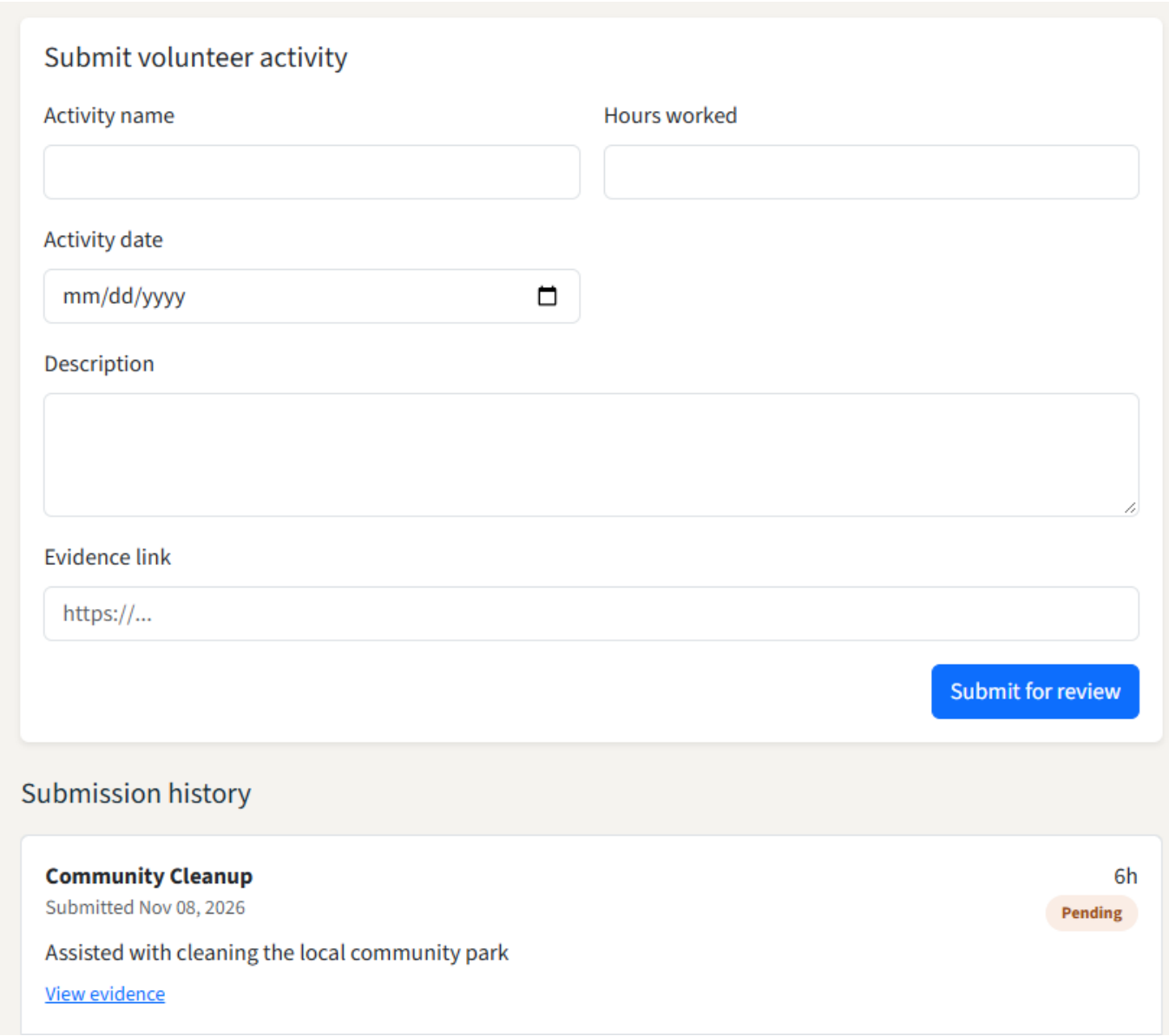



**Figure 8.** UAT01 - Updated screenshot showing the 6-hour Community Cleanup submission as pending.

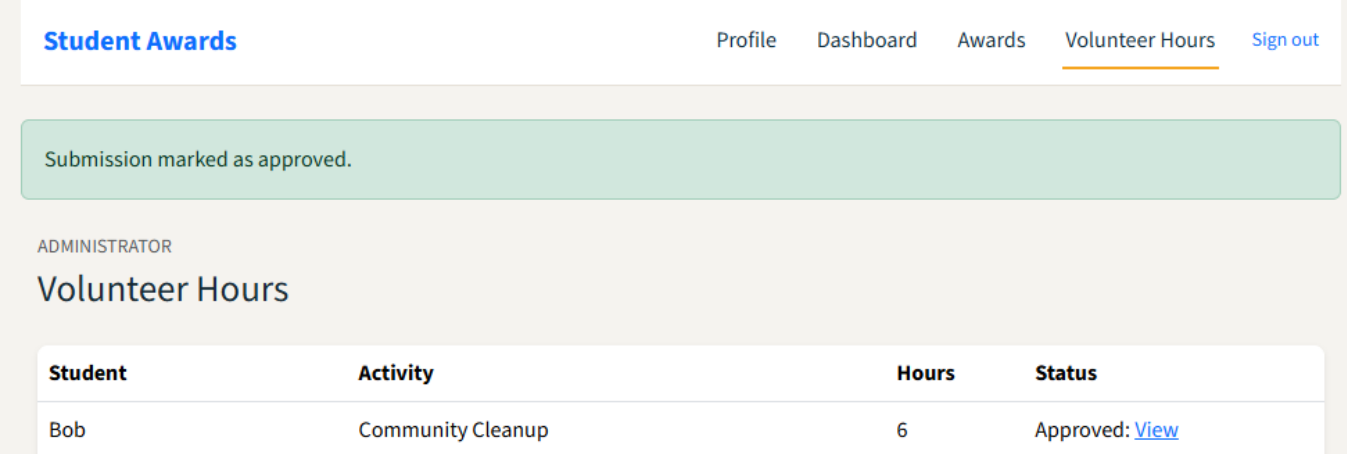



**Figure 9.** UAT02 - Administrator approval of the 6-hour Community Cleanup activity, matching the activity name and hours shown as pending in UAT01.

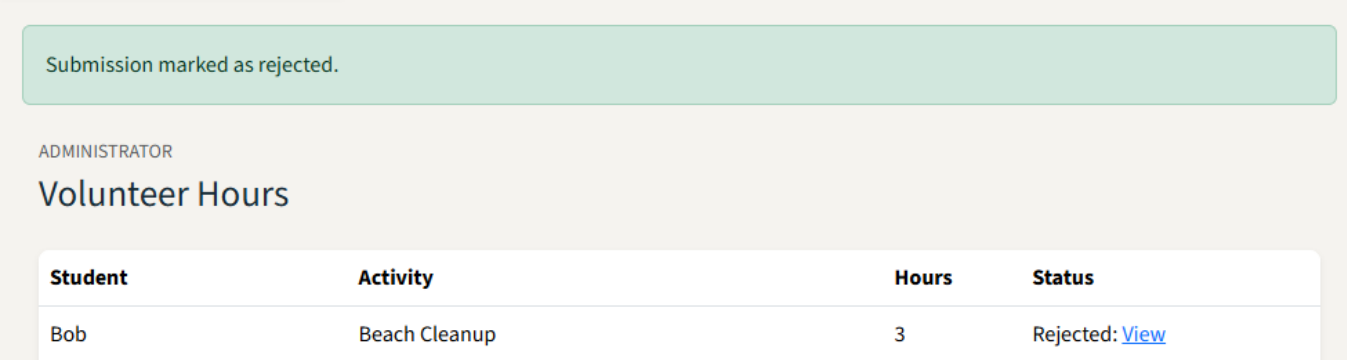



**Figure 10.** UAT03 - Administrator rejection confirmation and submission list.

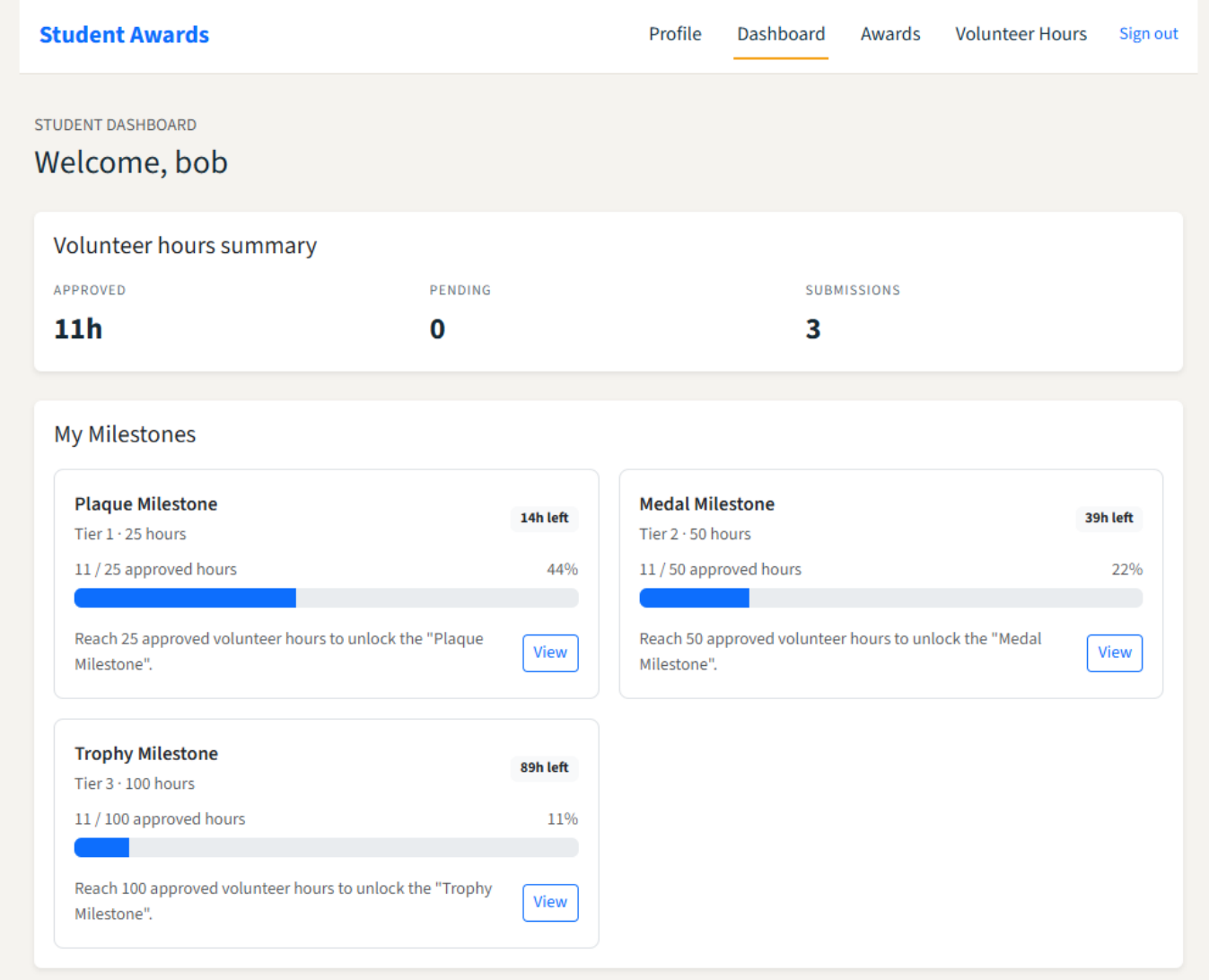



**Figure 11.** UAT04 - Student dashboard showing approved-hour progress.

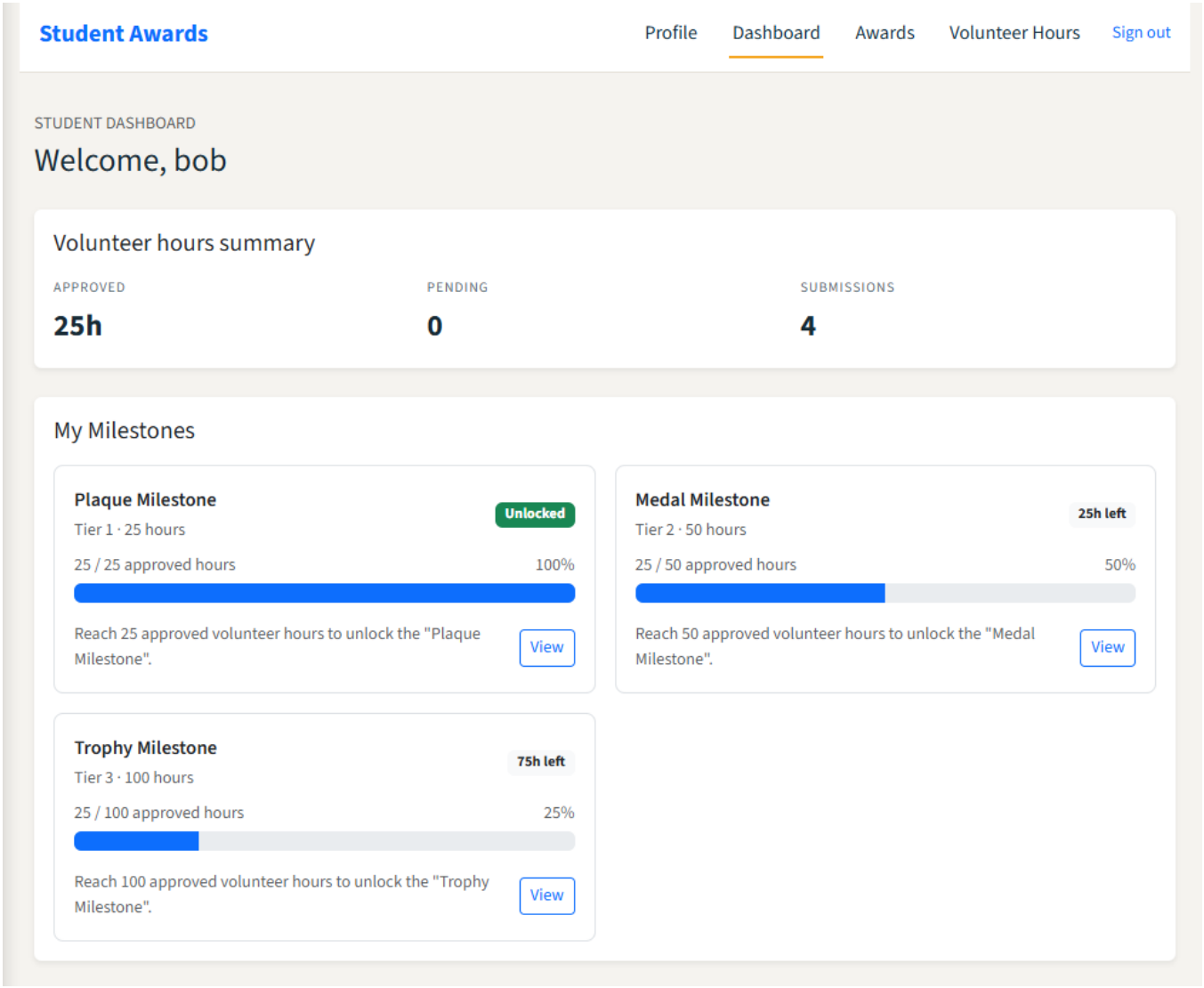



**Figure 12.** UAT05 - Dashboard showing the Plaque milestone unlocked.

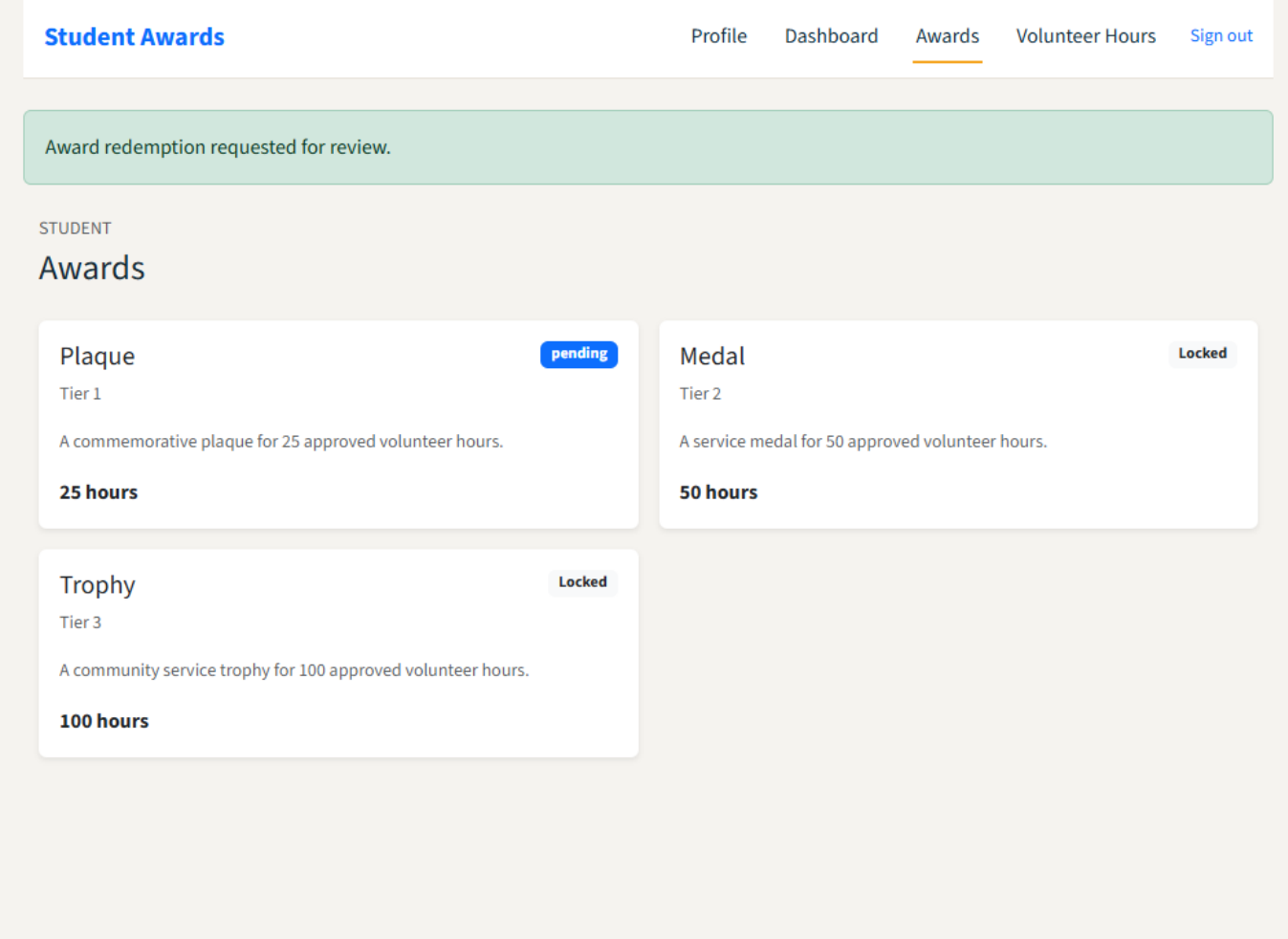



**Figure 13.** UAT06 - Award redemption request shown as pending.

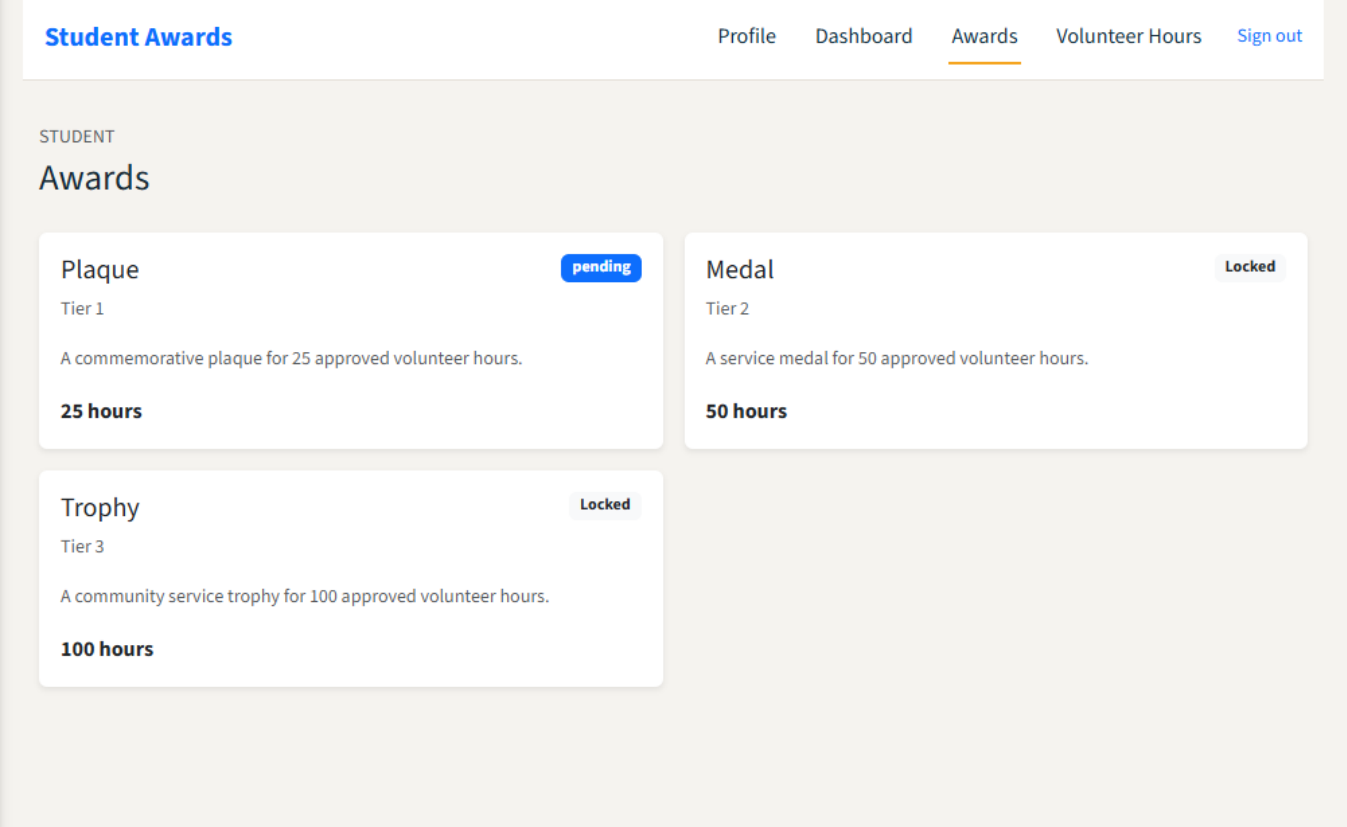



**Figure 14.** UAT07 - Existing pending redemption displayed with no second redemption action.

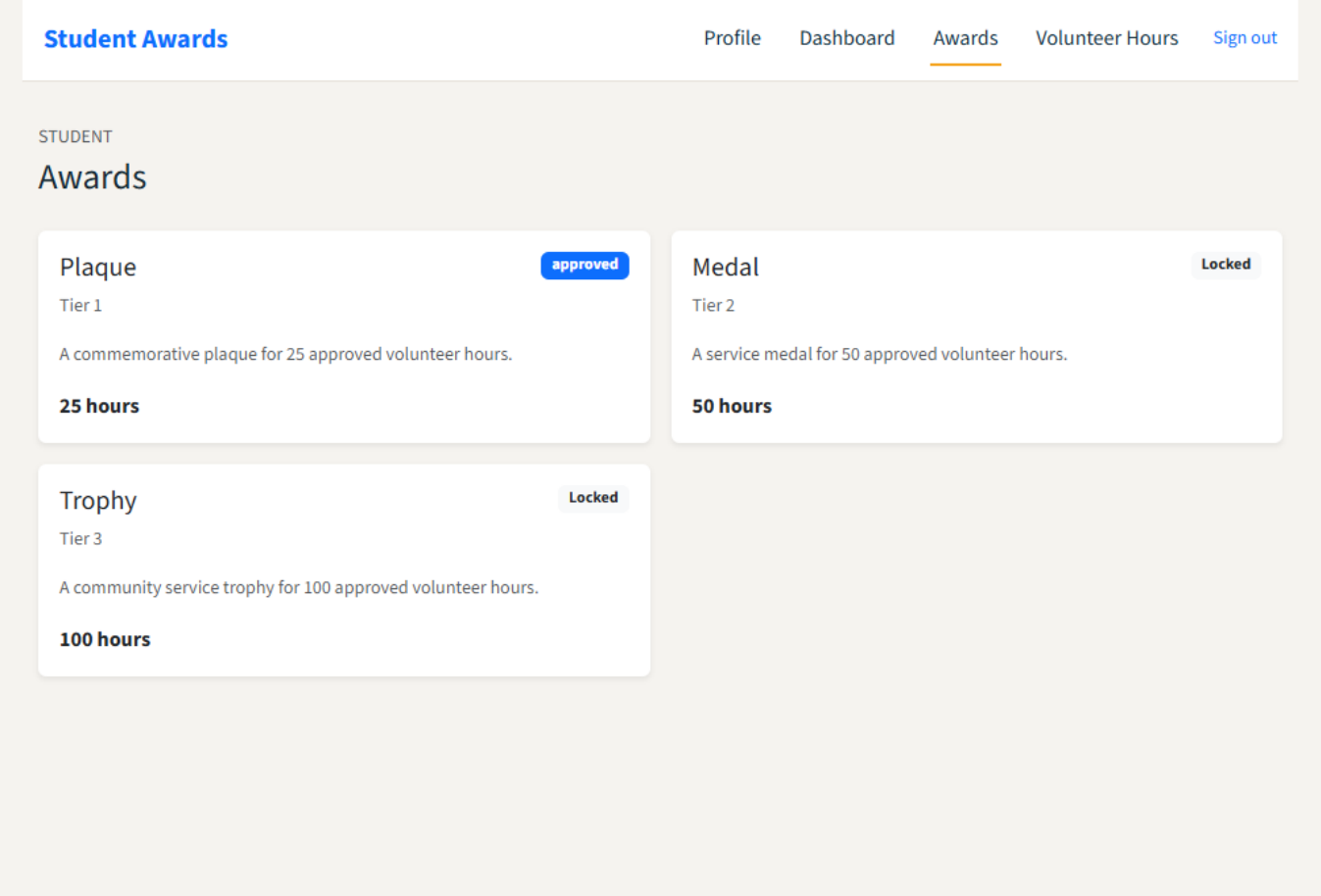



**Figure 15.** UAT08 - Administrator-approved award redemption status.

## F. Conclusion and limitations

The supplied test evidence reports **8/8 unit tests passed**, **7/7 integration tests passed**, **21/21 Postman assertions passed across 10 requests**, and **8/8 UAT cases passed**. The unit tests isolate service logic with mocks, while the integration tests use an in-memory SQLite database to exercise application components together. The Postman collection checks health, page responses, unauthenticated access, and an invalid milestone ID. The UAT plan and screenshots document the principal student and administrator workflows.

The results establish only the behaviors covered by these test cases and supplied evidence. The Postman collection is GET-only and does not exercise write endpoints or authenticated positive API workflows. The unit and integration test results do not replace browser acceptance testing, and this report does not claim that every route or edge case has been verified.

The updated UAT01 screenshot shows a **6-hour** Community Cleanup submission pending review, and UAT02 shows a **6-hour** Community Cleanup submission approved. The matching activity name and hour value are consistent with the review flow, although neither screenshot displays a submission ID to prove that they depict the same record. The UAT04 dashboard is an earlier progress snapshot at **11 approved hours**. After additional approved hours, the UAT05 screenshot shows **25 approved hours** and the Plaque milestone unlocked.

The integration-test screenshot contains one SQLite adapter deprecation warning. The recorded test run still passed; the warning is not evidence that the affected behavior failed, but it may merit attention in a future maintenance task.

## Appendix A. Test artifact locations

- Unit Test Plan: unit-test-plan.md
- Integration Test Plan: integration-test-plan.md
- User Acceptance Test Plan and completed results: uat-plan.md
- Unit tests: tests/test_volunteer_service_unit.py
- Integration tests: tests/test_student_awards_flow.py
- Postman collection: COMP3613 Assignment 2 - Student Awards API Tests.postman_collection.json